

Documento de Diseño Arquitectónico y Plan de Desarrollo: Ecosistema de Automatización Financiera Offline

Este plan de diseño exhaustivo estructura la construcción de tu plataforma basándose en la arquitectura de dos capas (Staging e Integración) y aprovecha a fondo la estructura interna de **Money Manager Ex (MMEX)**.

Fase 1: Capa de Extracción y Captura de Datos (Data Ingestion)

El objetivo de esta fase es obtener transacciones sin depender de la nube o conexiones permanentes, utilizando múltiples fuentes.

- **Pipeline Principal (Emails y Notificaciones):** Se construirá un script en Python que utilice el protocolo IMAP para capturar notificaciones bancarias en tiempo real 1. Para lidiar con el texto no estructurado o los formatos HTML, se implementarán **expresiones regulares (Regex)** combinadas con un LLM local (como **Ollama**) 1-3. Esto garantizará resiliencia si los bancos cambian sus plantillas de correo.
- **Pipeline de Contingencia (PDFs y Estados de Cuenta):** Para documentos estructurados, se utilizará un enfoque híbrido: bibliotecas de extracción de tablas deterministas como pdfplumber o Camelot, y herramientas de visión/OCR (Tesseract) como respaldo 2, 3.
- **Auditoría y Validación (La "Regla de Oro"):** Todo archivo procesado deberá pasar por una validación matemática obligatoria antes de ingresar al sistema. La suma de las transacciones extraídas debe reconciliar exactamente con el balance inicial y final reportado por el banco ($B_{\text{final}} = B_{\text{initial}} + \sum D - \sum W$) 4, 5. Si falla, el lote se marcará para revisión manual.

Fase 2: Capa de Staging y Motor ETL (Normalización)

Nunca se escribirá directamente en el archivo principal de MMEX sin una limpieza previa 6. Los datos crudos aterrizarán en staging.db (SQLite).

- **Normalización y Categorización:** Las fechas se estandarizarán a formato ISO. Para las descripciones comerciales de los bancos (que suelen ser confusas o variar), se aplicará **Fuzzy Matching con RapidFuzz** apoyado por un diccionario de reglas locales para asignar automáticamente la categoría y el beneficiario (ej. "AMZN MKTP" -> Beneficiario: "Amazon", Categoría: "Compras") 7-9.
- **Deduplicación (Hash FITID):** Debido a que la tabla principal de transacciones en MMEX (CHECKINGACCOUNT_V1) carece de una clave única natural que evite duplicados, el script ETL generará un **Hash SHA-256** utilizando la combinación de: *Fecha + Monto + Cuenta + Comercio* 6. Se aplicará un "sliding window" (ventana de tiempo) de aproximadamente 14 días para identificar y fusionar transacciones de correo que se asientan días después en el estado de cuenta 10.

Fase 3: Integración con MMEX y Diseño de Base de Datos

La transferencia de la capa de Staging a MMEX se realizará en dos etapas progresivas de madurez.

Etapas 3.1: Importación Vía CSV (MVP Recomendado)

- El motor ETL exportará los datos limpios a archivos CSV estandarizados 6.
- Dentro de MMEX, se configurará un "Preset" de importación libre (Universal CSV Importer) mapeando columnas exactas: *Date*, *Payee*, *Amount (+/-)*, *Category*, *Notes* 11-13. Esto delegará a MMEX la lógica de validación interna y el recálculo de balances 6, 14.

Etapas 3.2: Inyección Directa SQLite (Avanzado)

- El script en Python usará la librería `sqlite3` para ejecutar operaciones `INSERT INTO CHECKINGACCOUNT_V1` 15.
- **Protección Transaccional ACID:** Toda inyección a las tablas `PAYEE_V1`, `CATEGORY_V1` y `CHECKINGACCOUNT_V1` se envolverá estrictamente en bloques `BEGIN`, `COMMIT` y `ROLLBACK` para evitar corrupciones parciales si el script falla a la mitad 16, 17.
- **Gestión del Balance:** Al insertar directamente por SQLite, debes considerar que los *triggers* internos de MMEX pueden no actualizar automáticamente el saldo global en tiempo real. Tu script deberá encargarse de calcular matemáticamente el balance o invocar la lógica que asegura la integridad contable 15.

Fase 4: Implementación de "Custom Fields" (Campos Personalizados)

Para gestionar la deduplicación de la Etapa 3.2 sin ensuciar el campo de notas, implementarás **Campos Personalizados**.

- Crearás un campo personalizado de tipo *String* (Texto) destinado a alojar el identificador **Hash FITID** 18, 19.
- A nivel de base de datos, este hash se almacenará en la tabla secundaria `CUSTOMFIELDDATA_V1`, vinculándose al registro principal a través del `REFID` que corresponderá al `TRANSID` de la transacción 20, 21.
- **Ventaja Arquitectónica:** Al separar este metadato contable, podrás realizar validaciones de "Check before Insert" súper rápidas cruzando tu `staging.db` con `CUSTOMFIELDDATA_V1` para descartar transacciones que ya existen, logrando importaciones 100% libres de pérdidas 22, 23. Puedes mapear este campo al panel de vista (ej. `UDFC01`) para propósitos de auditoría 24.

Fase 5: Entorno Multi-usuario, Privacidad y Seguridad

Dado que el diseño es "Offline-First" y multi-usuario (familiar), la arquitectura de datos y redes se configurará de la siguiente manera:

- **Estructura del Archivo:** Se usará un único archivo de base de datos cifrado con AES-256 (extensión `.emb`), lo cual es indispensable al compartir el archivo a través de la red local o dispositivos 25-27.
- **Segregación de Datos:** En lugar de bases de datos separadas, se crearán `ACCOUNTID` únicos para "Mi tarjeta" y "Su tarjeta" 28. Para gastos compartidos, en lugar de duplicar categorías, se utilizará la tabla `TAG_V1` (Tags). Un gasto en supermercado podrá tener el tag "Gasto Conjunto" y otro el tag "Gasto Personal" 21, 29,

30. Esto facilita filtrar los reportes cruzados (Reportes -> Transacciones) usando condiciones lógicas (TagA | TagB) 31.

- **Sincronización de Red Local:** Para mantener ambos dispositivos sincronizados sin pasar por la nube pública, se utilizará **Syncthing** para compartir el directorio del archivo .emb 28, 32.
- **Prevención de Colisiones y Backups:** Al usar sincronización de archivos (en vez de una BD concurrente cliente-servidor), la concurrencia es el mayor riesgo 28, 33. Para evitar corrupción de bloqueos de archivo ("database is locked"), se configurará la creación de respaldos inmutables (.bak) cada vez que la base de datos se inicie y cierre 34, 35, permitiendo restauración inmediata si ocurren conflictos de escritura simultánea en Syncthing 36, 37. Se sugiere manejar copias locales temporales en móviles y computadoras para operaciones de gran volumen y dejar que Syncthing concilie las versiones finales 33, 38.